

Workshop

TDD, ADR & AI-Assisted Development

Datum: 20 maart 2026
Case study: Mithril — PWA gebouwd met Claude Code

Deel 1: Het Probleem — Context Rot

Wat is context rot?

AI coding assistants zoals Claude Code, GitHub Copilot en Cursor werken binnen een **context window** — een beperkt geheugen dat alleen de huidige conversatie bevat. Elke keer dat je een nieuwe chat start, begint de AI met een blanco geheugen.

Context rot is het fenomeen waarbij:

- Eerdere architectuurbeslissingen verloren gaan bij een nieuwe sessie
- De AI dezelfde fouten opnieuw maakt die je in een vorige chat al had gecorrigeerd
- Code inconsistent wordt omdat de AI niet meer weet welke patronen eerder gekozen zijn
- "Waarom is dit zo?" niet meer te beantwoorden is — niet door jou, niet door de AI

Waarom is dit een groot probleem?

Bij traditionele development zit de context in het hoofd van de developer. Bij AI-assisted development is de AI je "pair programmer" — maar eentje met geheugenverlies. Stel je voor dat je elke ochtend een nieuwe collega krijgt die:

- Niet weet welke afspraken er vorige week zijn gemaakt
- Niet begrijpt waarom bepaalde technische keuzes zijn gemaakt
- Dezelfde vragen stelt die je gisteren al hebt beantwoord
- Code schrijft die inconsistent is met wat er al staat

In **Mithril** zou dit betekenen: 318 commits, maar bij elke nieuwe chat een risico op tegenstrijdige patronen, vergeten constraints, en dubbel werk.

De oplossing: externe context die de sessie overleeft

De AI mag dan zijn geheugen verliezen, maar bestanden in je repository niet. De sleutel is om context te externaliseren in drie lagen:

Laag	Wat het vastlegt	Waar het leeft
Tests (TDD)	Gedrag en contracten	`tests/`
ADRs	Architectuurbeslissingen en de <i>*waarom*</i>	`logs/decisions/`
PRDs	Product requirements: <i>*wat*</i> en <i>*waarom*</i> op feature-niveau	`plans/`
Plans	Technische scope, specs en voortgang	`plans/`

Opmerking: Sinds de oorspronkelijke versie van deze workshop is het systeem uitgebreid met **Product Requirements Documents (PRDs)**. PRDs formaliseren de functionele eisen en het *waarom* op feature-niveau, vóórdat er een technisch plan wordt geschreven. Ze vormen de schakel tussen een idee en een implementatieplan, en voorkomen dat de AI (of een developer) begint te bouwen zonder helder gedefinieerde scope en doelen.

Deel 2: Test-Driven Development met AI

Waarom TDD essentieel is bij AI-assisted development

TDD is geen "nice to have" bij AI-assisted development — het is een **veiligheidsnet dat voorkomt dat de AI je codebase sloopt**.

Zonder TDD:

- De AI schrijft code die "klopt" in isolatie, maar bestaande functionaliteit breekt
- Je merkt regressies pas veel later (of helemaal niet)
- Bij elke nieuwe sessie heb je geen garantie dat de AI de bestaande contracten respecteert

Met TDD:

- Tests beschrijven het verwachte gedrag — de AI leest ze en begrijpt de contracten
- Regressies worden direct gevangen
- De test suite is een levende specificatie die elke nieuwe sessie overleeft

Het Red-Green-Refactor principe Case study: HasFollowUp trait bug (ADR-001)

Situatie: Tijdens het schrijven van tests voor de `HasFollowUp` trait ontdekte de TDD-cyclus een design bug die bij normale development onopgemerkt was gebleven.

De trait definieerde scopes (``scopeOverdue``, ``scopeDueToday``) die direct kolommen ``follow_up_date`` en ``status`` bevroegen. Maar die kolommen bestaan alleen op de ``follow_ups`` tabel — niet op ``tasks`` of ``team_members`` die de trait gebruiken. **Runtime SQL errors gegarandeerd.**

Hoe TDD dit ving:

- RED** — Tests geschreven voor ``Task::withOverdueFollowUps()`` scope
- Test faalde** — niet omdat de implementatie fout was, maar omdat het hele design fout was
- Beslissing** — Scopes herschreven naar ``whereHas``-based queries (gedocumenteerd in ADR-001)
- GREEN** — Nieuwe implementatie passeert alle tests
- Resultaat** — Bug gevonden voordat er ook maar een regel productcode was geschreven

Les: De tests vonden de bug. Niet de AI, niet code review, niet een QA-engineer. De test faalde, en dat dwong een redesign af voordat de fout in productie kon belanden.

De TDD skill in de praktijk

De ``/bdk-tdd`` skill dwingt de red-green-refactor cyclus af bij elke implementatie:

Belangrijk: De skill vertelt de AI expliciet dat een test die meteen slaagt verdacht is — het test dan bestaand gedrag (overbodig) of test niet wat je denkt (fout).

Mithril in cijfers

Metric	Waarde
Testbestanden	161
Testfuncties	995
Commits	318
Ratio tests/commits	~3:1

Elke nieuwe feature begon met tests. Elke bug fix begon met een test die de bug reproduceerde.

Deel 3: Architecture Decision Records

Wat is een ADR?

Een ADR documenteert:

Context — Wat was het probleem? Welke alternatieven zijn overwogen?

Decision — Wat is er besloten en waarom?

Consequences — Wat verandert er? Welke impact heeft dit?

Waarom ADRs cruciaal zijn bij AI-assisted development

Zonder ADRs kan een AI:

Een beslissing terugdraaien die je bewust hebt genomen

Een alternatief voorstellen dat je al hebt afgewezen (met goede redenen)

Dezelfde trade-off discussie opnieuw voeren in elke sessie

Met ADRs:

De AI leest de bestaande ADRs aan het begin van elke sessie

Eerdere beslissingen worden gerespecteerd

Nieuwe beslissingen worden automatisch gedocumenteerd

Case study: Recurring Tasks en het Event Dispatch Gap (ADR-013)

Situatie: Bij het implementeren van recurring tasks ontdekte de TDD-cyclus dat ``TaskStatusChanged`` events alleen werden gefired vanuit expliciete dispatch calls. Maar ``AutoSaveController``, ``bulkUpdate()`` en ``move()`` updaten status via ``$task->update()`` *zonder* het event te dispatchen.

Consequentie zonder ADR: In een volgende sessie zou de AI niet weten dat:

Er een ``TaskObserver`` is toegevoegd als oplossing

Handmatige event dispatches bewust zijn verwijderd

Dit systeem ook het pre-bestaande probleem met ``CreateFollowUpOnWaiting`` oploste

Met ADR-013: Elke toekomstige sessie leest dit record en begrijpt het complete plaatje — de trigger, de alternatieven, en de gekozen oplossing.

Case study: FK Removal als bewuste keuze (ADR-019)

Situatie: ``attachments.activity_id`` had een FK constraint met ``cascadeOnDelete``. Maar het plan specificeerde ook een cleanup command voor orphaned attachments — die structureel onmogelijk zouden zijn met een FK cascade.

Drie alternatieven overwogen (gedocumenteerd in de ADR):

FK houden, cleanup command verwijderen

FK houden, cleanup command anders testen

FK verwijderen, cleanup command behouden

Zonder ADR zou een toekomstige AI (of developer) de missende FK als een bug beschouwen en hem "fixen". **De ADR voorkomt dat een bewuste beslissing wordt teruggedraaid.**

Het ADR-systeem in de praktijk

Het systeem bestaat uit drie skills:

``/bdk-adr`` — Activatie aan het begin van elke sessie:

- Leest het architectuurplan
- Leest de ADR-index en relevante bestaande ADRs
- Bevestigt de scope van het werk met bekende constraints

``/bdk-adr-entry`` — Automatisch getriggerd wanneer:

- De implementatie afwijkt van het plan
- Een API contract of datamodel verandert
- Een technologie wordt toegevoegd of vervangen
- Een niet-triviale trade-off wordt gemaakt

Vuistregel: Als iemand later zou kunnen vragen "waarom is dit zo?", schrijf een ADR.

Mithril ADRs: 23 records in 3 weken

Van simpele trait redesigns (ADR-001) tot complexe integratiebeslissingen (ADR-008: Office 365 Calendar Integration).
Elke ADR volgt hetzelfde template:

Deel 4: Het Plan-systeem — De lijm ertussen

Waarom een plan?

Tests beschrijven **wat** het systeem doet. ADRs beschrijven **waarom** keuzes zijn gemaakt. Maar geen van beide beschrijft **wat je aan het bouwen bent en hoe ver je bent**.

Het plan is het overkoepelende document dat:

- De scope definieert (en expliciet vastlegt wat **out of scope** is)
- Het werk opdeelt in fases die in een sessie passen
- Elke spec testbaar maakt (directe input voor TDD)
- Voortgang bijhoudt (checkboxes per spec item)

`/bdk-plan-create` — Van idee naar gestructureerd plan

Het proces:

Cruciale regel: *"Specs must be testable. Every spec item should be verifiable with a test."*

Dit is waar plan en TDD samenkomen — elke bullet in het plan wordt een test in de red-green-refactor cyclus.

`/bdk-plan-execute` — Gestructureerde uitvoering

Het uitvoeringsproces integreert alle drie de systemen:

Voorbeeld: Recurring Tasks plan

Let op Phase 0 — die stond niet in het oorspronkelijke plan. Tijdens TDD werd het event dispatch gap ontdekt, het plan werd bijgewerkt, en een ADR geschreven (ADR-013). Het plan is een levend document.

Deel 5: Hoe de drie systemen samenwerken

De feedback loop Wat dit oplost

Probleem	Oplossing
Context rot (AI vergeet vorige sessie)	ADRs + plan overleven sessies
Regressies (AI breekt bestaande code)	Tests vangen regressies direct
Inconsistentie (AI gebruikt andere patronen)	Plan + codebase scan afdwingen patronen
Scope creep (AI voegt features toe)	Plan met "Out of Scope" sectie
Teruggedraaide beslissingen (AI "fixt" bewuste keuzes)	ADRs documenteren het *waarom*
Dubbel werk (AI lost opgelost probleem opnieuw op)	ADR-index geeft overzicht

Het belang van het *waarom*

De meest waardevolle informatie in een ADR is niet *wat* er is besloten, maar *waarom*. Vergelijk:

Zonder ADR: "attachments.activity_id heeft geen FK constraint"

Volgende developer/AI: "Dat is een bug, laat me even een FK toevoegen..."

Met ADR-019: "FK bewust verwijderd zodat de cleanup command testbaar is en edge cases vangt"

Volgende developer/AI: "Ah, dat is een bewuste keuze. Ik ga hier niet aankomen."

Deel 6: Praktische tips & valkuilen (10 min)

Valkuilen bij AI-assisted development

"De AI weet het toch" — Nee. Elke sessie begint leeg. Als het niet in een bestand staat, bestaat het niet.

Grote sessies — Context windows hebben limieten. Lange sessies leiden tot context compressie waarbij nuances verloren gaan. Kleine fases in het plan houden sessies beheersbaar.

Geen tests = geen vangnet — Zonder TDD merk je niet dat de AI in sessie 5 iets heeft gebroken dat in sessie 2 werkte.

ADRs achteraf schrijven — Dan mis je de alternatieven die je hebt overwogen. Schrijf ADRs op het moment van de beslissing.

Plan niet bijwerken — Een plan dat niet de werkelijkheid reflecteert is erger dan geen plan. Houd het actueel.

AI blind vertrouwen — De AI is een krachtig hulpmiddel, maar jij blijft de architect. Review, denk mee, stel vragen.

Tips

Start elke sessie met context laden — In Mithril activeert `CLAUDE.md` automatisch `/bdk-tdd`, `/bdk-adr` en `/bdk-lemp` bij elke nieuwe chat. Dit kan met elke AI-assistent: geef hem bij de start de relevante bestanden.

Maak specs testbaar — Als je het niet in een test kunt uitdrukken, is de spec te vaag.

Houd ADRs klein en focused — Een ADR per beslissing, niet een mega-document per feature.

Gebruik het plan als communicatiemiddel — Het plan is niet alleen voor de AI — het is ook voor jou en je team. Een goedgekeurd plan voorkomt discussie tijdens implementatie.

Externaliseer alles — Context die alleen in je hoofd zit, is context die verloren gaat. Schrijf het op.

Samenvatting

Component	Functie	Overleeft sessies?
TDD	Beschrijft gedrag, vangt regressies	Ja (tests in repo)
ADR	Documenteert beslissingen + waarom	Ja (markdown in repo)
PRD	Formaliseert feature-eisen en doelen	Ja (markdown in repo)
Plan	Definieert technische scope, fases, specs	Ja (markdown in repo)
CLAUDE.md	Laadt context automatisch bij start	Ja (config in repo)

De kracht zit in de combinatie: de PRD definieert **wat** er gebouwd moet worden en **waarom**, het plan vertaalt dat naar technische specs, TDD verifieert **dat** het correct is, en ADRs leggen vast **waarom** technische keuzes zijn gemaakt. Samen vormen ze een systeem dat context rot voorkomt en consistente, betrouwbare AI-assisted development mogelijk maakt.

Mithril bewijst het: 995 tests, 23 ADRs, 12 plannen, 318 commits — gebouwd in 3 weken, met consistente architectuur ondanks tientallen sessiewisselingen.

Discussie / Q&A

Mogelijke discussiepunten:

- Hoe past dit in jullie huidige workflow?
- Welke onderdelen zijn direct toepasbaar zonder AI?
- Hoe schaal je dit naar een team met meerdere developers?
- Wat is de overhead vs. de tijdsbesparing?